DATA SCIENCE – ACTIVITY

Source code:

1.Extract and load your dataset:

```
import zipfile
import os
import pandas as pd
zip_path = "/content/archive (4).zip"
with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall("/content/traffic_dataset")
csv_path = "/content/traffic_dataset/smart_city_traffic_mobility.csv"
df = pd.read_csv(csv_path)
print("Dataset uploaded and loaded successfully!")
print("Dataset shape:", df.shape)
display(df.head())
```

2. Check and analyze the dataset:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
print("Dataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nFirst 5 Records:")
display(df.head())
```

```
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDataset Information:")
df.info()
```

3. **Analyze congestion levels:**

```
print("Congestion Level Distribution:")
print(df["congestion_level"].value_counts())
plt.figure(figsize=(8, 5))
df["congestion_level"].value_counts().plot(kind="bar")
plt.title("Traffic Congestion Level Distribution")
plt.xlabel("Congestion Level")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

4. **Prepare the data for machine learning:**

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
target = "congestion_level"
# Remove identifiers and columns that could reveal the target
excluded = [
```

```
    "record_id", "road_id", "intersection_id",
    "timestamp", "congestion_level", "congestion_score",
    "traffic_density", "queue_length", "average_wait_time",
    "emission_estimate", "fuel_waste_estimate"
]
X = df.drop(columns=excluded, errors="ignore")
y = df[target]
# Identify numerical and categorical columns
numeric_cols = X.select_dtypes(include=["number", "bool"]).columns.tolist()
categorical_cols = X.select_dtypes(
    include=["object", "category"]
).columns.tolist()
print("Numerical Features:", numeric_cols)
print("Categorical Features:", categorical_cols)
# Handle missing values and encode text features
preprocessor = ColumnTransformer([
    ("numeric", SimpleImputer(strategy="median"), numeric_cols),
    ("categorical", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols)
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)
print("\nTraining records:", len(X_train))
print("Testing records:", len(X_test))
```

5. **Train the Random Forest model:**

```
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1,
        class_weight="balanced"
    ))
])
print("Training started...")
model.fit(X_train, y_train)
print("Model trained successfully!")
```

6. **Evaluate model performance:**

```
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

```
print("MODEL EVALUATION RESULTS")
print("-----------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

7. **Plot the confusion matrix:**

```
labels = model.named_steps["classifier"].classes_
cm = confusion_matrix(y_test, y_pred, labels=labels)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=labels
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap="Blues", values_format="d")
plt.title("Traffic Congestion Prediction")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()
```

8. **Display actual and predicted results:**

```
results = pd.DataFrame({
    "Actual Congestion": y_test.values,
    "Predicted Congestion": y_pred
})
print("Sample Predictions:")
display(results.head(20))
```

```
print("\nActual vs Predicted Counts:")
display(pd.crosstab(
    results["Actual Congestion"],
    results["Predicted Congestion"]
))
```

Output:

1.Extract and load your dataset:

```
Dataset uploaded and loaded successfully!
Dataset shape: (204000, 47)
```

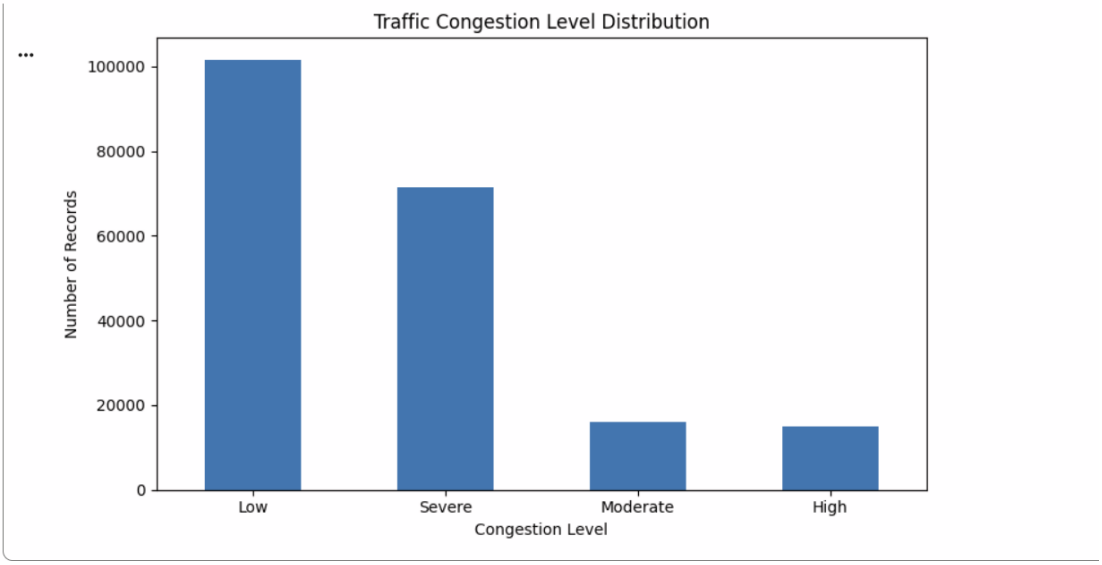
	record_id	city_zone	road_id	intersection_id	latitude	longitude	road_type	lanes	speed_limit	vehicle_count	...	traffic_camera_count	iot_sensor_health	signal_c
0	REC_0000001	Financial District	RD_206	INT_001	40.757551	-73.990504	Local Street	2	40	162	...	1	Optimal	
1	REC_0000002	Financial District	RD_206	INT_001	40.757551	-73.990504	Local Street	2	40	150	...	1	Optimal	
2	REC_0000003	Financial District	RD_206	INT_001	40.757551	-73.990504	Local Street	2	40	186	...	1	Optimal	
3	REC_0000004	Financial District	RD_206	INT_001	40.757551	-73.990504	Local Street	2	40	162	...	1	Optimal	
4	REC_0000005	Financial District	RD_206	INT_001	40.757551	-73.990504	Local Street	2	40	168	...	1	Optimal	

5 rows × 47 columns

2. Check and analyze the dataset:

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204000 entries, 0 to 203999
Data columns (total 47 columns):
 #   Column                  Non-Null Count   Dtype
---  ------                  --------------   -----
 0   record_id               204000 non-null  object
 1   city_zone               204000 non-null  object
 2   road_id                 204000 non-null  object
 3   intersection_id         204000 non-null  object
 4   latitude                204000 non-null  float64
 5   longitude               204000 non-null  float64
 6   road_type               204000 non-null  object
 7   lanes                   204000 non-null  int64
 8   speed_limit             204000 non-null  int64
 9   vehicle_count           204000 non-null  int64
 10  average_speed           204000 non-null  float64
 11  heavy_vehicle_count     204000 non-null  int64
 12  motorcycle_count        204000 non-null  int64
 13  public_transport_count  204000 non-null  int64
 14  traffic_density         204000 non-null  float64
 15  queue_length            204000 non-null  float64
 16  average_wait_time       204000 non-null  float64
 17  timestamp               204000 non-null  object
 18  hour                    204000 non-null  int64
 19  day_of_week             204000 non-null  int64
 20  is_weekend              204000 non-null  int64
 21  is_holiday              204000 non-null  int64
```

3. **Analyze congestion levels:**



4. **Prepare the data for machine learning:**

```
Numerical Features: ['latitude', 'longitude', 'lanes', 'speed_limit', 'vehicle_count', 'average_speed', 'heavy_vehicle_count', 'motorcycle_count', 'public_transport_count',
Categorical Features: ['city_zone', 'road_type', 'peak_period', 'weather_condition', 'road_condition', 'iot_sensor_health']

Training records: 163200
Testing records: 40800
```

5. **Train the Random Forest model:**

```
Training started...
Model trained successfully!
```

6. **Evaluate model performance:**

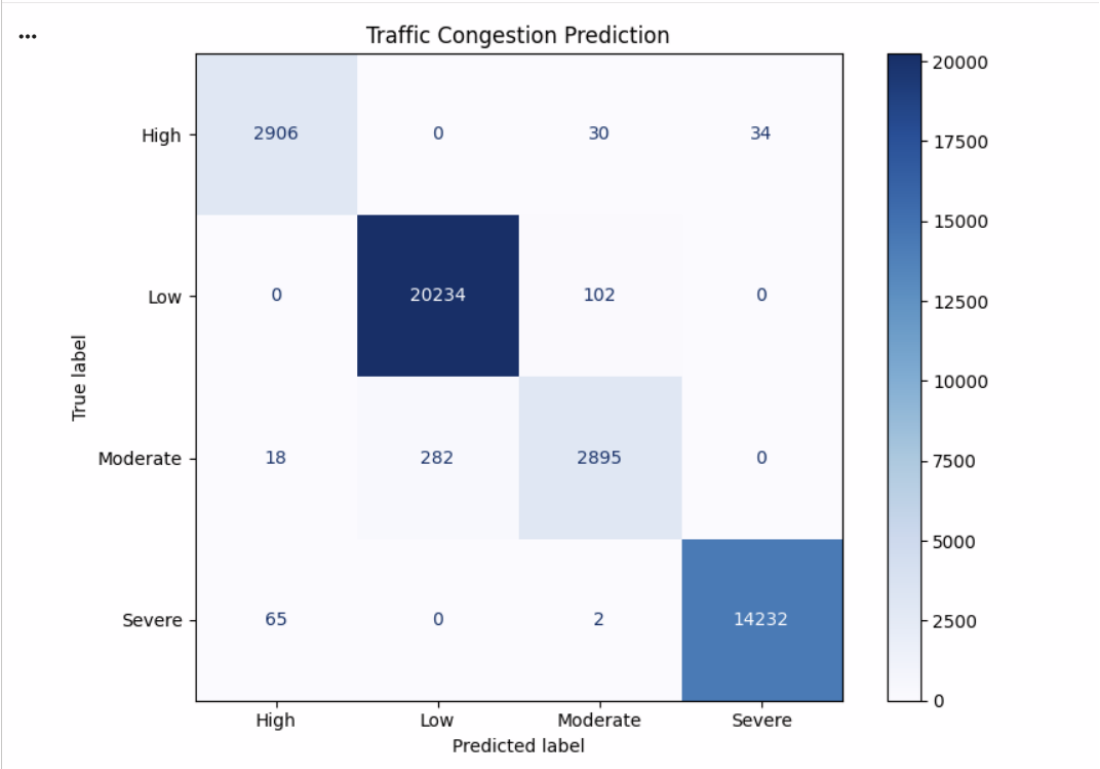
```
MODEL EVALUATION RESULTS
------------------------
Accuracy: 98.69 %

Classification Report:
              precision    recall  f1-score   support

        High       0.97      0.98      0.98      2970
         Low       0.99      0.99      0.99     20336
    Moderate       0.96      0.91      0.93      3195
      Severe       1.00      1.00      1.00     14299

    accuracy                           0.99     40800
   macro avg       0.98      0.97      0.97     40800
weighted avg       0.99      0.99      0.99     40800
```

7. **Plot the confusion matrix:**



8. **Display actual and predicted results:**

Sample Predictions:

	Actual Congestion	Predicted Congestion
0	Severe	Severe
1	Low	Low
2	Low	Low
3	Severe	Severe
4	Moderate	Moderate
5	Severe	Severe
6	Severe	Severe
7	Low	Low
8	Low	Low
9	High	High
10	High	High
11	Low	Low
12	Low	Low
13	Low	Low
14	Low	Low